

CS 4530: Fundamentals of Software Engineering

Module 09: Concurrency Patterns in Typescript

Adeel Bhutta, Rob Simmons and Mitch Wand
Khoury College of Computer Sciences

Learning Goals for this Lesson

- At the end of this lesson, you should be prepared to:
 - Explain the difference between JS run-to-completion semantics and interrupt-based semantics.
 - Given a simple program using `async/await`, work out the order in which the statements in the program will run.
 - Write simple programs that create and manage promises using `async/await`
 - Write simple programs to mask latency with concurrency by using non-blocking IO and `Promise.all` in TypeScript.

Why async matters

- Your browser probably spends 99% of its time waiting
 - for http requests to return
 - for database calls to return
- Wouldn't it be nice if we could use that waiting time for something else?
 - maybe we have some other http request we could launch
 - maybe other things (throbbers??)
- **async** is Typescript's way of handling this

Lesson 1: Basic terminology

Typescript types distinguish functions that take a long time to return

- An ordinary function might have type

```
(arg:number) => number;
```

- A long-executing function will have type

```
(arg:number) => Promise<number>;
```

A promise is a thing that if you **await** it, you get back a value

```
import { addAsync } from "../src/Slides/myFetch";  
// addAsync returns a promise for the sum of x and y
```

```
async function slowAdd(x:number,y:number): Promise<number> {  
  const p : Promise<number> = addAsync(x,y);  
  const result : number = await p;  
  return result;  
}
```

src/Slides/slowAdd.ts

Promise-returning functions are created with **async**.

```
import { addAsync } from "../src/Slides/myFetch";  
// addAsync returns a promise for the sum of x and y
```

```
async function slowAdd(x:number,y:number): Promise<number> {  
    const p : Promise<number> = addAsync(x,y);  
    const result : number = await p;  
    return result;  
}
```

Wait! Where did **slowAdd** return a promise?

```
import { addAsync } from "../src/Slides/myFetch";  
// addAsync returns a promise for the sum of x and y
```

```
async function slowAdd(x:number,y:number): Promise<number> {  
    const p : Promise<number> = addAsync(x,y);  
    const result : number = await p;  
    return result;  
}
```

Hang on, we'll get back to that later.

```
import { addAsync } from "../src/Slides/myFetch";  
// addAsync returns a promise for the sum of x and y
```

```
async function slowAdd(x:number,y:number): Promise<number> {  
  const p : Promise<number> = addAsync(x,y);  
  const result : number = await p;  
  return result;  
}
```

If your function calls **await** then your function is long-running, too

```
import { addAsync } from "../src/Slides/myFetch";  
// addAsync returns a promise for the sum of x and y
```

```
async function slowAdd(x:number,y:number): Promise<number> {  
  const p : Promise<number> = addAsync(x,y);  
  const result : number = await p;  
  return result;  
}
```

...which means that your function must return a promise.

```
import { addAsync } from "../src/Slides/myFetch";  
// addAsync returns a promise for the sum of x and y
```

```
async function slowAdd(x:number,y:number): Promise<number> {  
    const p : Promise<number> = addAsync(x,y);  
    const result : number = await p;  
    return result;  
}
```

Your async function will usually await one or more promises

```
export async function make3SequentialRequests() {  
  console.log('making 3 sequential requests');  
  const result1 = await makeRequest(1);  
  const result2 = await makeRequest(2);  
  const result3 = await makeRequest(3);  
  console.log('all 3 requests done, results:', [result1, result2,  
result3]);  
}
```

```
// await timeit(make3ConcurrentRequests);  
await timeit(make3SequentialRequests);
```

It will return a promise to complete the first promise that it awaits...

```
export async function make3SequentialRequests() {  
  console.log('making 3 sequential requests');  
  const result1 = await makeRequest(1);  
  const result2 = await makeRequest(2);  
  const result3 = await makeRequest(3);  
  console.log('all 3 requests done, results:', [result1, result2,  
result3]);  
}
```

```
// await timeit(make3ConcurrentRequests);  
await timeit(make3SequentialRequests);
```

src/Slides/requestPatterns.ts

...and go on from there

```
export async function make3SequentialRequests() {  
  console.log('making 3 sequential requests');  
  const result1 = await makeRequest(1);  
  const result2 = await makeRequest(2);  
  const result3 = await makeRequest(3);  
  console.log('all 3 requests done, results:', [result1, result2, result3]);  
}
```

src/Slides/requestPatterns.ts

But where do promises come from in the first place?

```
import { addAsync } from "../src/Slides/myFetch";  
// addAsync returns a promise for the sum of x and y
```

```
async function slowAdd(x:number,y:number): Promise<number> {  
  const p : Promise<number> = addAsync(x,y);  
  const result : number = await p;  
  return result;  
}
```

src/Slides/slowAdd.ts



Answer: JS/TS has an API for generating primitive promises

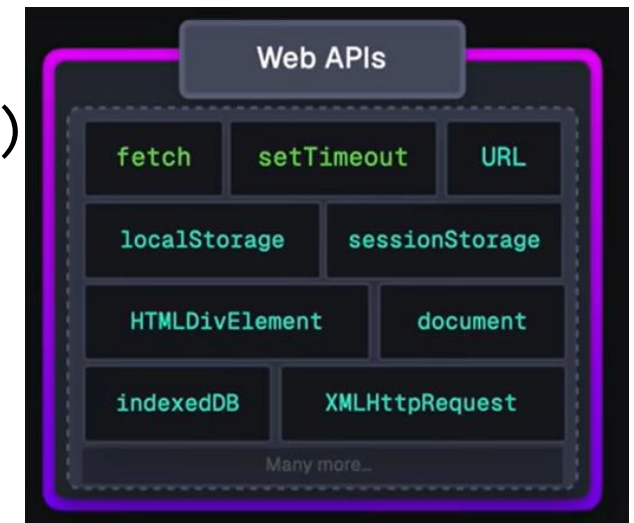
- These are things like http requests, I/O operations, or timers.
- Each of these returns a promise that you can **await**. The promise runs while it is pending, and produces the response from the http request, or the contents of the file, etc.
- You will hardly ever call one of these primitives yourself; usually they are wrapped in a convenient procedure, e.g., we write

```
axios.get('https://rest-example.covey.town')
```

to make an http request, or

```
fs.readFile(filename)
```

to read the contents of a file.



Second important property: Promises run to completion

- This means that the code you write can't be interrupted by other processes.
- In OS terms, this means that most of your code resides in critical sections.
- These critical sections are delimited in statically-determinable places.
- Let's look at some examples to see what all that means.

A Typical Example

src/Slides/runABC.ts

```
async function A(n: number) {  
  console.log(`A1 ${n}`);  
  await B(n);  
  console.log(`A2 ${n}`);  
  console.log('====');  
}
```

```
async function B(n: number) {  
  console.log(`B1 ${n}`);  
  await C(n);  
  console.log(`B2 ${n}`);  
}
```

```
async function C(n: number) {  
  console.log(`C1 ${n}`);  
  await myFetch(`ABC ${n}`);  
  console.log(`C2 ${n}`);  
}
```

\$ node runABC.ts

running runABC.ts

A1 1

B1 1

C1 1

Promise created for ABC 1

C2 1

B2 1

A2 1

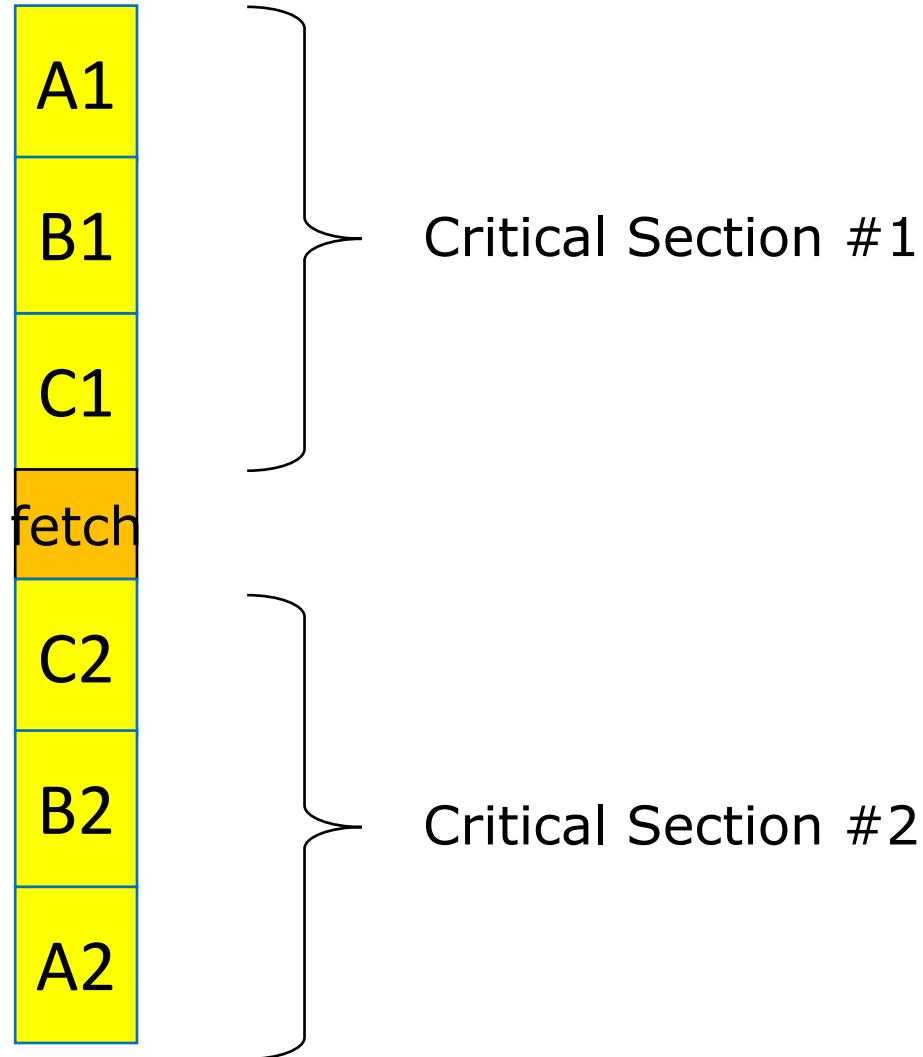
=====

A Typical Example

```
async function A(n: number) {  
  console.log(`A1 ${n}`);  
  await B(n);  
  console.log(`A2 ${n}`);  
  console.log('====');  
}
```

```
async function B(n: number) {  
  console.log(`B1 ${n}`);  
  await C(n);  
  console.log(`B2 ${n}`);  
}
```

```
async function C(n: number) {  
  console.log(`C1 ${n}`);  
  await myFetch(`ABC ${n}`);  
  console.log(`C2 ${n}`);  
}
```



What happens when we try to run two promises concurrently?

```
import A from './abc.ts';  
import X from './xyz.ts';  
  
await Promise.all([A(10), X(20)]);
```

src/Slides/runABCvsXYZ.ts

Here's what the result demonstrates.

```
$ node runABCVsXYZ.ts
```

```
A1 10
```

```
B1 10
```

```
C1 10
```

```
Promise created for ABC 10
```

```
X1 20
```

```
Y1 20
```

```
Z1 20
```

```
Promise created for XYZ 20
```

```
C2 10
```

```
B2 10
```

```
A2 10
```

```
=====
```

```
Z2 20
```

```
Y2 20
```

```
X2 20
```

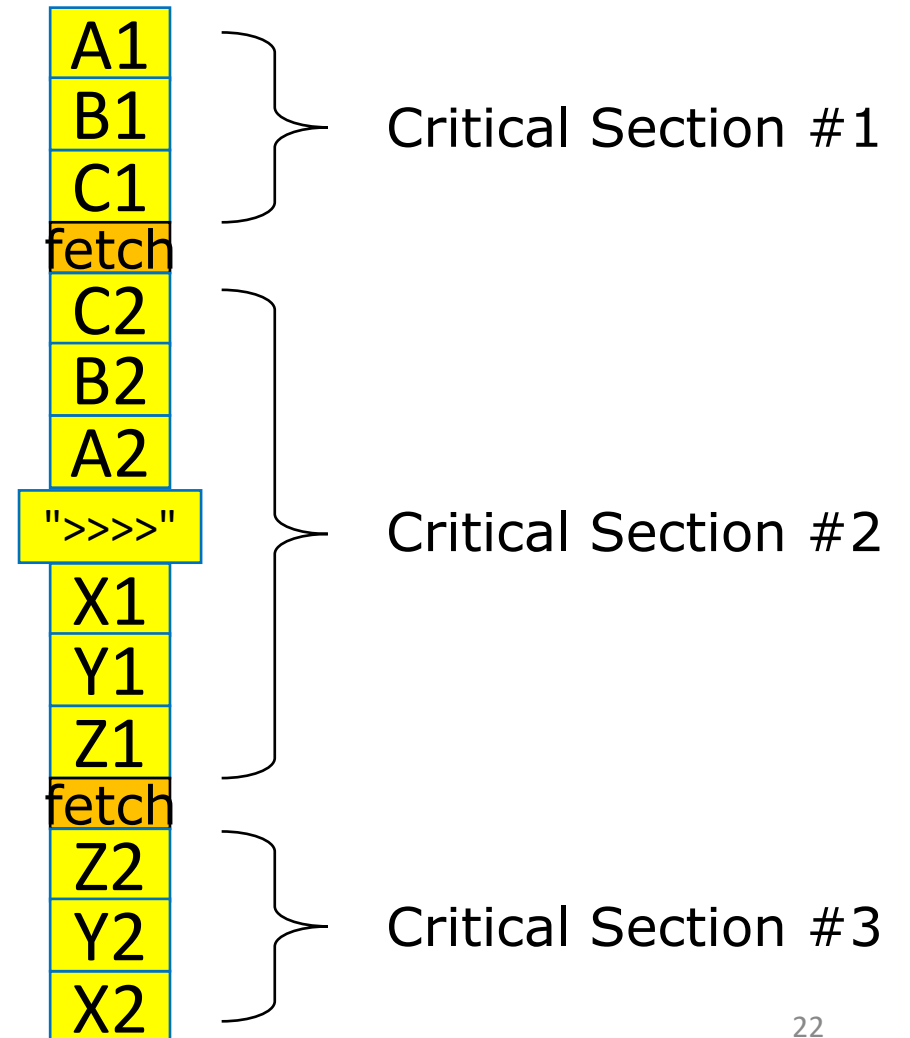
```
=====
```

- Running both A and X concurrently
- Illustrates critical sections: ABC and CBA always occur together without interleaving
- And similarly for XYZ and ZYX.
- Promises are created in order
- But completion order is non-deterministic: sometimes the run ends with ABC before XYZ, sometimes the reverse

You can have more than one await in an async function

```
async function main () {  
  await A(10);  
  console.log('>>>>>');  
  await X(20);  
}  
  
void main();
```

```
$ node abcThenXyz.ts  
A1 10  
B1 10  
C1 10  
Promise created for ABC 10  
C2 10  
B2 10  
A2 10  
====  
>>>>>  
X1 20  
Y1 20  
Z1 20  
Promise created for XYZ 20  
Z2 20  
Y2 20  
X2 20  
====
```



src/Slides/abcThenXYZ.ts

This is called **cooperative multiprocessing**

- In cooperative multiprocessing, **only one process is executed at a time.**
- **Each process pauses when it is convenient** to allow other processes to make progress.
- To make this practical (and avoid cheating!), we need a programming model that encourages this behavior
- This is what `async/await` provides

Most OS's use **pre-emptive multiprocessing**

- OS manages multiprocessing with multiple threads of execution
- Processes may be ***interrupted*** at unpredictable times
- Inter-process communication by shared memory
- Data races abound
- Really, really hard to get right: need critical sections, semaphores, monitors (all that stuff you learned about in op. sys.)

Lesson 2: Using async

Pattern for starting a concurrent computation using non-blocking I/O

```
export async function makeRequest(requestNumber:number) {  
  console.log(`starting makeRequest(${requestNumber})`);  
  const response = await fakeRequest(requestNumber);  
  console.log('request:', requestNumber, '\nresponse:', response)  
  return response;  
}
```

1. The first console.log is printed
2. The http request is sent, using non-blocking i/o
3. The browser goes about its business
4. Eventually, the http request responds and **response** gets its value, the console.log is printed, and makeRequest returns the value of response to its caller.

Use Promise.all to execute several requests concurrently

src/Slides/requestPatterns.ts

```
// make 3 concurrent requests using Promise.all
export async function make3ConcurrentRequests() {
  console.log('making 3 concurrent requests');
  const results = await Promise.all([
    makeRequest(1),
    makeRequest(2),
    makeRequest(3)
  ]);
  console.log('all 3 requests done, results: [ 101, 102, 103 ]');
}
```

```
await timeit(make3ConcurrentRequests);
```

```
$ node requestPatterns.ts
making 3 concurrent requests
starting makeRequest(1)
starting makeRequest(2)
starting makeRequest(3)
request: 1 returned 101
request: 3 returned 103
request: 2 returned 102
all 3 requests done, results: [ 101, 102, 103 ]
828.12 msec
```

If you add awaits, the requests will be processed sequentially

```
export async function make3SequentialRequests() {
  console.log('making 3 sequential requests');
  const result1 = await makeRequest(1);
  const result2 = await makeRequest(2);
  const result3 = await makeRequest(3);
  console.log('all 3 requests done, results: [',
    result1, result2, result3, ']');
}
```

```
// await timeit(make3ConcurrentRequests);
await timeit(make3SequentialRequests);
```

```
$ node requestPatterns.ts
making 3 sequential requests
starting makeRequest(1)
request: 1 returned 101
starting makeRequest(2)
request: 2 returned 102
starting makeRequest(3)
request: 3 returned 103
all 3 requests done, results: [ 101, 102, 103 ]
1222.06 msec
```

If you add awaits, the requests will be processed sequentially

```
export async function make3SequentialRequests() {
  console.log('making 3 sequential requests');
  const result1 = await makeRequest(1);
  const result2 = await makeRequest(2);
  const result3 = await makeRequest(3);
  console.log('all 3 requests done, results: [',
    result1, result2, result3, ']');
}
```

```
// await timeit(make3ConcurrentRequests);
await timeit(make3SequentialRequests);
```

```
$ node requestPatterns.ts
making 3 sequential requests
starting makeRequest(1)
request: 1 returned 101
starting makeRequest(2)
request: 2 returned 102
starting makeRequest(3)
request: 3 returned 103
all 3 requests done, results: [ 101, 102, 103 ]
1222.06 msec
```

...but it would be much slower

```
$ tsx timeComparison.ts  
After 100 runs of length 10 with delay 100ms  
makeRequestsConcurrently: min = 107   avg = 109 max = 115 msec  
makeRequestsSerially      : min = 1085  avg = 1093 max = 1103 msec
```

Why is that?

Visualizing Promise.all

Sequential (await)

“Don’t make another request until you got the last response back”

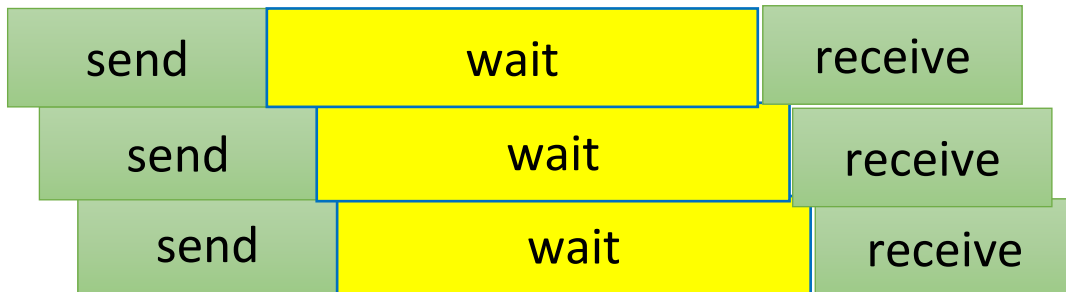
237 msec



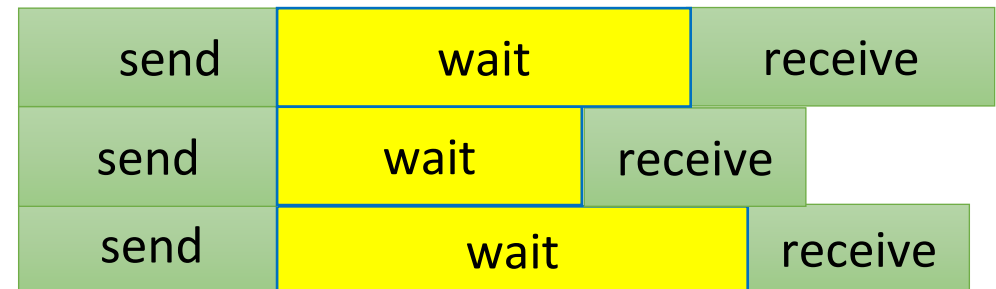
Concurrent (Promise.all)

“Make all of the requests now, then wait for all of the responses”

34 msec



or



Requests can also be chained (if they are serial)

```
async function main() {  
  console.log('main started');  
  const request1 = 32  
  const res1 = await fakeRequest(request1);  
  console.log(`fakeRequest(${request1}) returned`);  
  const res2 = await fakeRequest(res1);  
  console.log(`fakeRequest(${res1}) returned`);  
  const res3 = await fakeRequest(res2);  
  console.log(`fakeRequest(${res2}) returned`);  
  console.log([request1, res1, res2, res3]);  
  console.log('main done');  
}
```

```
$ node requestPatterns.ts  
making 3 chained requests  
starting makeRequest(1)  
request: 1 returned 101  
starting makeRequest(101)  
request: 101 returned 201  
starting makeRequest(201)  
request: 201 returned 301  
all 3 requests done, results: [ 101, 201, 301 ]  
1306.77 msec
```

src/Slides/requestPatterns.ts

Use try/catch to deal with errors

```
// a request that may fail
async function maybeFailingRequest(req: number): Promise<number> {
  const res = await fakeRequest(req);
  if (res < 0) {
    throw new Error(`Request ${req} failed because response ${res} < 0`);
  } else {
    return res;
  }
}
```

src/Slides/tryCatchExample.ts

try/catch, continued

```
async function main() {  
  console.log('main started');  
  const req1 = -32  
  let res: number;  
  try {  
    res = await maybeFailingRequest(req1);  
    console.log(`fakeRequest(${req1}) returned: ${res}`);  
  } catch (err) {  
    console.error(`Error occurred for request ${req1}`);  
    res = 0  
  }  
  console.log('main done with res =',  
}
```

```
timeIt(main);
```

```
$ npx ts-node tryCatchExample.ts  
main started  
fakeRequest received request: -32  
time passes....  
Error occurred for request -32  
main done with res = 0  
1007.52 msec
```

Pattern for testing an async function

```
import { fakeRequest } from './fakeRequest.ts'
import { describe, expect, test } from 'vitest'
```

src/Slides/example.test.ts

```
// fakeRequest should make a network request using myFetch
// and then returns n + 1
```

```
describe('fakeRequest', () => {

  test('fakeRequest should return its argument + 1 (await/expect)', async () => {
    expect.assertions(1)
    await expect(fakeRequest(100)).resolves.toEqual(101)
  })

  test('fakeRequest should return its argument + 1 (expect/await)', async () => {
    expect.assertions(1)
    expect(await fakeRequest(7)).toEqual(8)
  })
})
```

AntiPattern 1: **unawaited** promise

```
// fakeRequest(n) is an async that waits for 1 second and then  
// resolves with the number n+10  
import { fakeRequest } from "../fakeRequest";  
import { timeIt } from "../timeIt";
```

```
async function main() {  
  console.log('main started');  
  const request = 32  
  const res = fakeRequest(request);  
  console.log(`fakeRequest(${request})`);  
  console.log('main done');  
}
```

```
timeIt(main)
```

```
$ npx ts-node oneRequestNoAwait.ts  
main started: 32  
fakeRequest(32) returned: [object Promise]  
main done  
2.64 msec  
fakeRequest received request: 32  
time passes....
```

What just happened?

```
$ npx ts-node oneRequestNoAwait.ts  
main started  
fakeRequest(32) returned: [object Promise]  
main done  
2.64 msec  
fakeRequest received request: 32  
time passes....
```

1. `main()` called `fakeRequest(32)`.
2. `fakeRequest(32)` created a unit of work (a Promise), and told the runtime to run it sometime or other.
3. Normally, we wouldn't see the actual value returned by `fakeRequest(32)`, because we'd just wait for the unit of work to run before proceeding.
4. But here, we didn't wait-- we just took the value returned by `fakeRequest(32)`-- the Promise-- and printed it.
5. We finished our current critical section, printing "main done", which informed the runtime that we were done.
6. The runtime then looked around for another critical section to start. In this case, it found the critical section created by `fakeRequest(32)`, and ran it, printing the last two lines

Wow! That was complicated!

- We try to make our code easy to understand.
- That's why it's an **anti**pattern.
- Luckily, in real code we don't need to do this very often

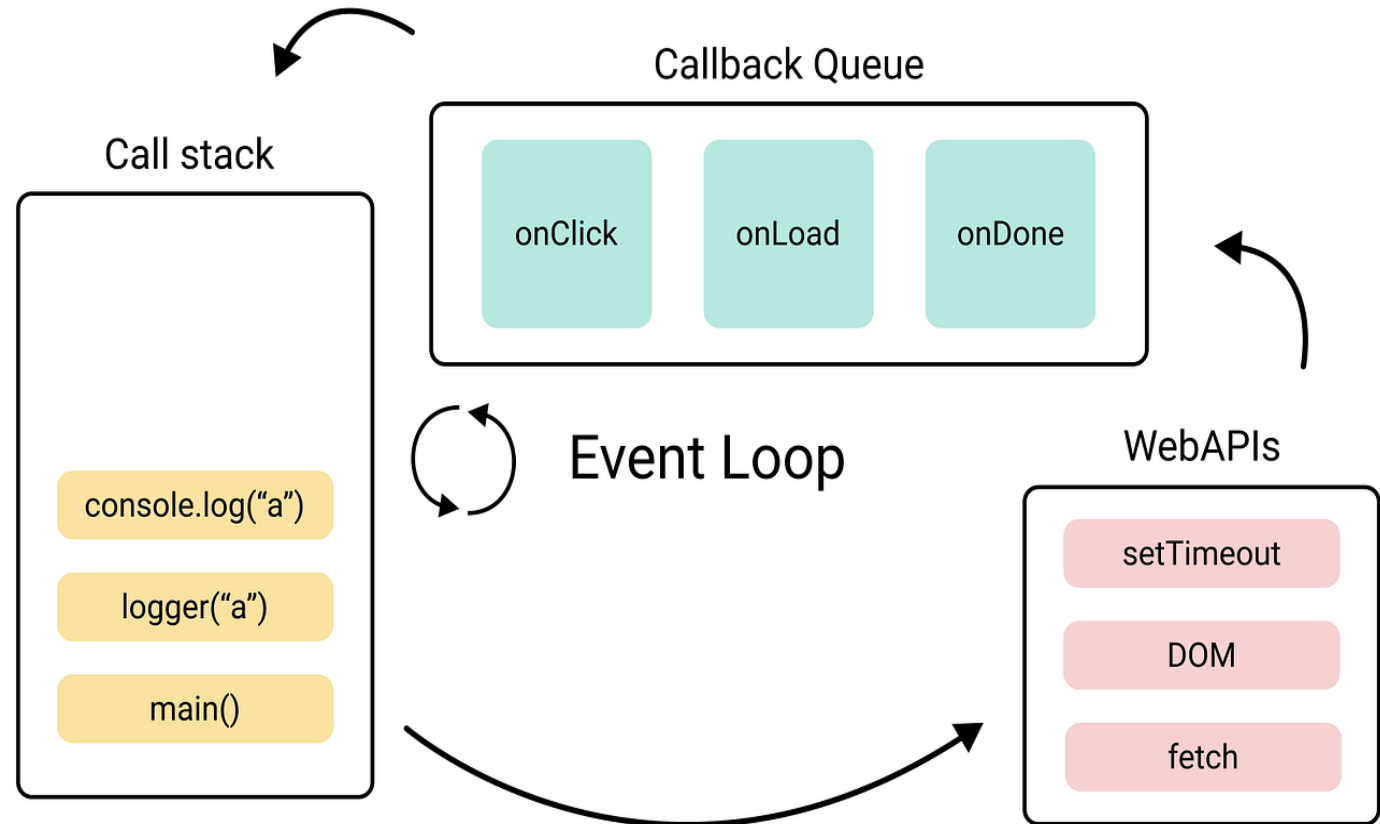
General Rules for Writing Asynchronous Code

- You can't return a value from a promise to an ordinary procedure.
 - You can only return the value to the async that is awaiting it.
- Call async procedures only from other async functions or from the top level.
- Break up any long-running computation into **async/await** segments so other processes will have a chance to run.
- Leverage concurrency when possible
 - Use **promise.all** if you need to wait for multiple promises to return (or **Promise.allSettled**, if needed).
- Check for errors with **try/catch**

Section 3: Odds and Ends

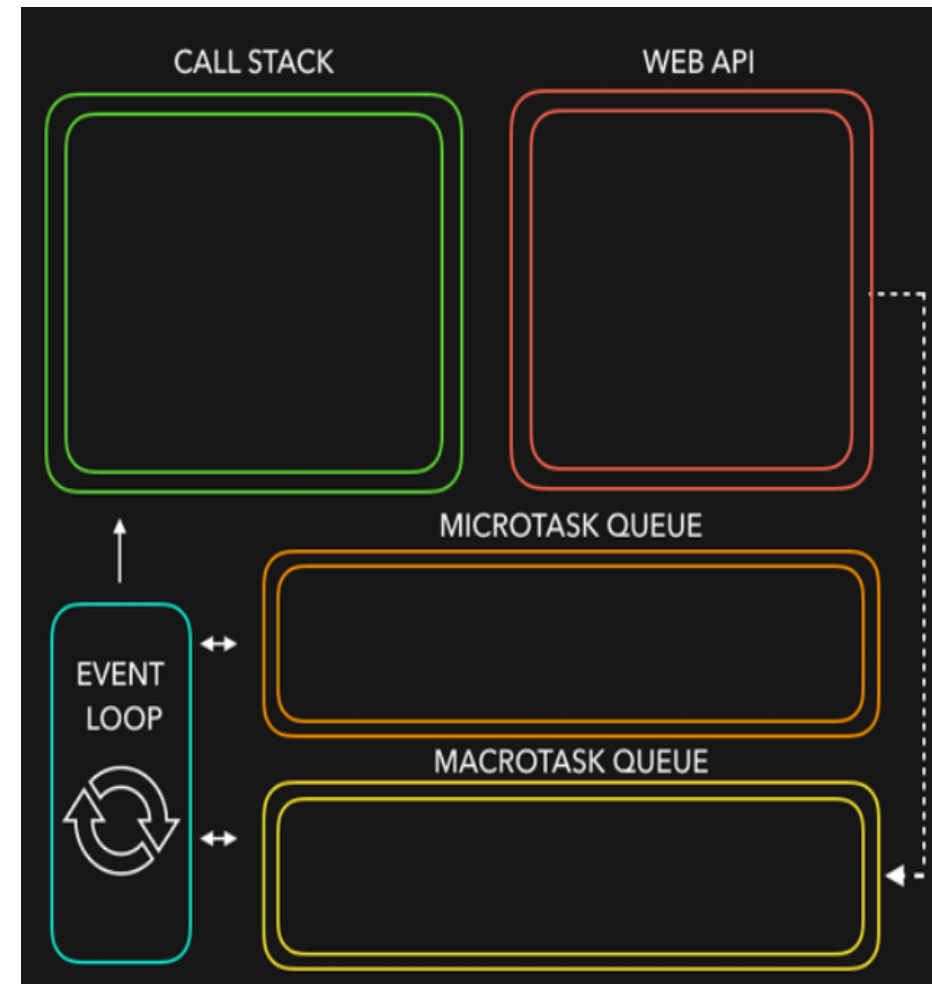
How does JS Engine make this happen?

- One Event Loop means that we have single thread of execution
- WebAPI is used for asynchronous tasks
- Queues are used for "await"-ing tasks
- When call stack gets empty, event loop picks up tasks from Callback Queue



Let's put it all together

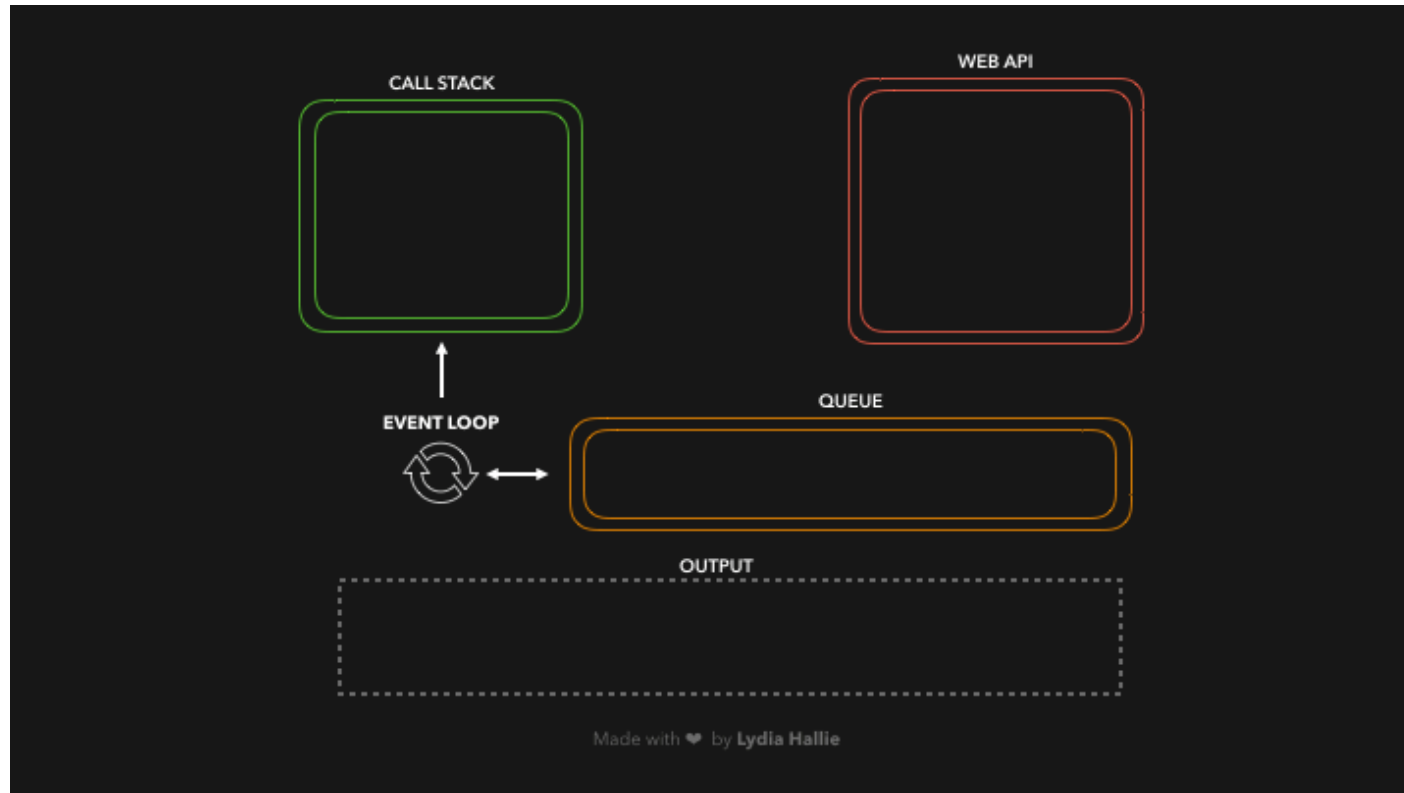
- JS/TS has single event loop
- We outsource most of the non-blocking IO work (to WebAPIs) for asynchronous work
- Upon completion, they are placed in queues (Microtask queue has priority over Macrotask queue)
- Event loop picks them up from queue when call stack is empty!



Here is a quick demo for you

```
const foo = () => console.log("First");
const bar = () => setTimeout(() => console.log("Second"), 500);
const baz = () => console.log("Third");

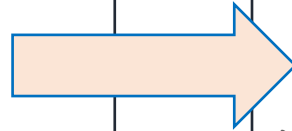
bar();
foo();
baz();
```



Courtesy of <https://dev.to/lydiahallie/javascript-visualized-event-loop-3dif>

Async/await code is compiled into promise/then code

```
async function
makeThreeSerialRequests() {
1.  console.log('Making first
request');
2.  await makeOneGetRequest();
3.  console.log('Making second
request');
4.  await makeOneGetRequest();
5.  console.log('Making third
request');
6.  await makeOneGetRequest();
7.  console.log('All done!');
}
makeThreeSerialRequests();
```



```
console.log('Making first request');
makeOneGetRequest().then(( ) =>{
    console.log('Making second request');
    return makeOneGetRequest();
}).then(( ) => {
    console.log('Making third request');
    return makeOneGetRequest();
}).then(( )=>{
    console.log('All done!');
});
```

Promises Enforce Ordering Through “Then”

```
1. console.log('Making requests');
2. axios.get('https://rest-example.covey.town/')
   .then((response) =>{
       console.log('Heard back from server');
       console.log(response.data);
   });
3. axios.get('https://www.google.com/')
   .then((response) =>{
       console.log('Heard back from Google');
   });
4. axios.get('https://www.facebook.com/')
   .then((response) =>{
       console.log('Heard back from Facebook');
   });
5. console.log('Requests sent!');
```

- **axios.get** returns a promise.
- **p.then** mutates that promise so that the then block is not run until after the original promise returns.
- The resulting promise isn't completed until the then block finishes.
- You can chain **.then**'s, to get things that look like `p.then().then().then()`
- Each **then** is a pause point.

async doesn't eliminate data races

```
import { myFetch } from './fakeRequest.ts';

const log: string[] = [];

async function X(n: number) {
  await myFetch(1)
  log.push('X');
}

async function Y(n: number) {
  await myFetch(2)
  log.push('Y');
}

for (let i = 0; i < 4; i += 1) {
  log.push('start');
  await Promise.all([X(10), Y(20)]);
}

console.log(log);
```

```
$ node dataRace.ts
Promise created for 1
Promise created for 2
Promise created for 1
Promise created for 2
Promise created for 1
Promise created for 2
Promise created for 1
Promise created for 2
[
  'start', 'X', 'Y',
  'start', 'Y', 'X',
  'start', 'Y', 'X',
  'start', 'Y', 'X'
]
```

What a returned promise really looks like

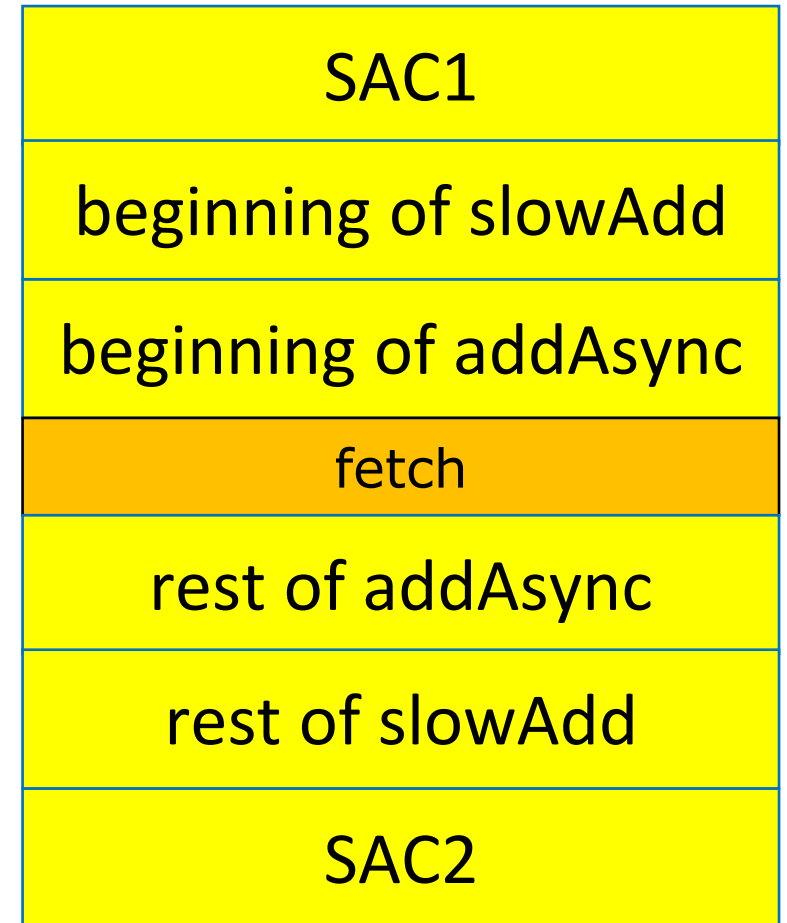
```
import { addAsync } from "../src/Slides/myFetch";  
// addAsync returns a promise for the sum of x and y
```

```
async function slowAdd(x:number,y:number): Promise<number> {  
    const p : Promise<number> = addAsync(x,y);  
    const result : number = await p;  
    return result;  
}
```

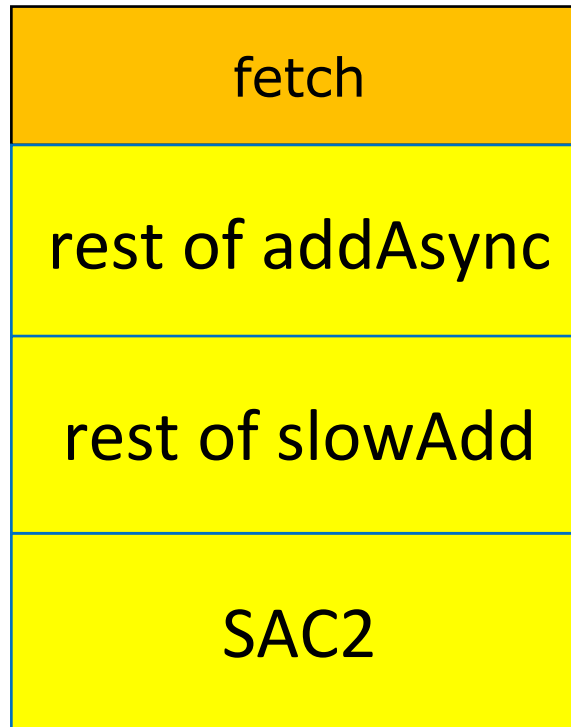
Let's look at a somewhat richer example

```
export async function slowAddCaller(x:number,y:number):  
  Promise<number> {  
    console.log(`SAC1 ${x},${y}`)  
    const result : number = await slowAdd(x,y);  
    console.log(`SAC2 ${result}`)  
    return result;  
  }
```

```
void (await slowAddCaller(3,4));
```



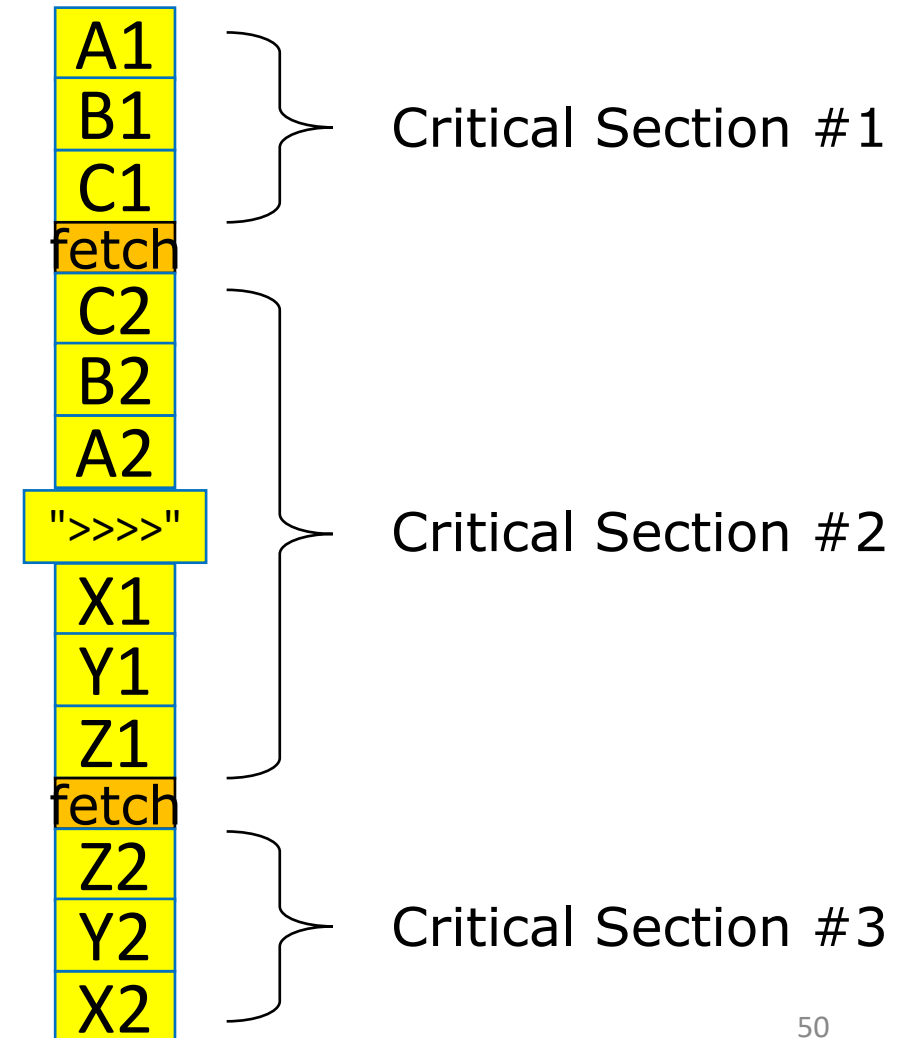
fetch creates a promise to finish the whole chain of async calls.



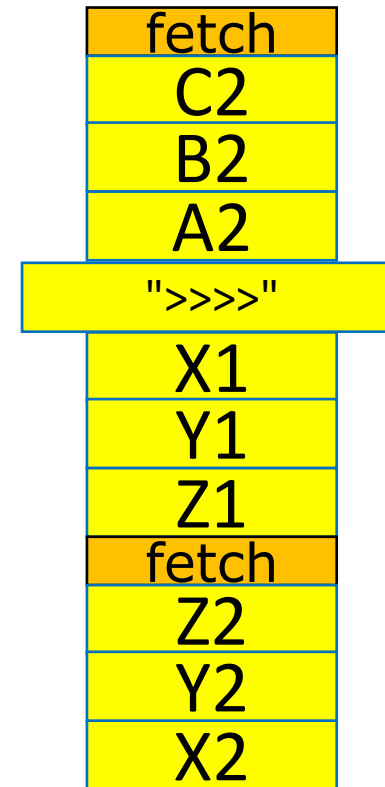
Remember this?

```
async function main () {  
  await A(10);  
  console.log('>>>>');  
  await X(20);  
}
```

```
void main();
```



The promise returned by A is everything after the first fetch



Review

- You should now be prepared to:
 - Explain the difference between JS run-to-completion semantics and interrupt-based semantics.
 - Given a simple program using `async/await`, work out the order in which the statements in the program will run.
 - Write simple programs that create and manage promises using `async/await`
 - Write simple programs to mask latency with concurrency by using non-blocking IO and `Promise.all` in TypeScript.